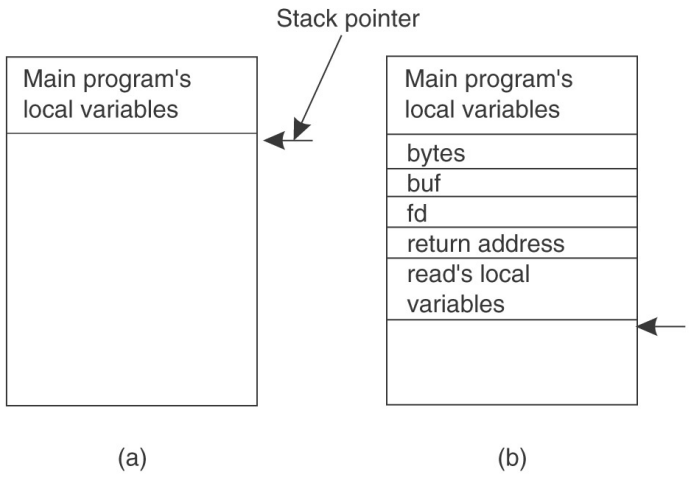
RPC – Remote Procedure Call
Apel de procedura conventional

Fie apelul unei functii **read** de citire a unui sir de caractere dintr-un fisier:
count = read (fd, buf, bytes)
unde fd este descriptorul fisierului din care se citesc caracterele
buf este untablou de caractere in care sunt plasate datele citite
bytes spune cate caractere sa se citeasca, iar
count este rezultatul, care arata cate caractere s-au citit efectiv.
Executia are loc astfel:
programul principal apeleaza read, care este probabil o rutina de biblioteca;
aceasta face un apel sistem (OS) al unei functii de citire echivalente
OS umple buf cu caracterele citite si intoarce rezultatul catre functia read care, la randul sau, il va returna programului principal.
Parametrii, in C, pot fi apelati prin **valoare** (de exemplu **fd** si **bytes**) sau prin **referinta** (de exemplu **buf**) cu efectele cunoscute: parametrul valoare este folosit pentru a transmite o valoare de la programul apelant la procedura, in timp ce parametrul referinta este folosit pentru a transmite o valoare (sau mai multe) de la procedura la variabila referita din programul apelant.

Comunicarea parametrilor

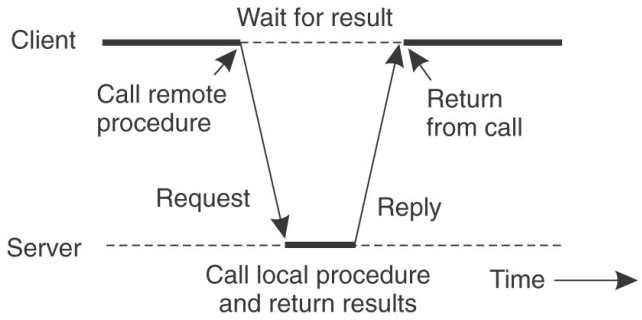
Comunicarea valorilor parametrilor se face prin stiva. Figura urmatoare arata continutul stivei inainte de apelul lui read (a) si in timpul executiei acestei proceduri (b). La apel, programul principal pune pe stiva valorile parametrilor (incepand cu ultimul din lista) si adresa de intoarcere. Ordinea parametrilor este utla procedurii pentru identificare valorii fiecarui parametru.



La terminarea executiei procedurii read, rezultatul este pus intr-un registru, adresa de revenire si variabilele locale sunt inlaturate din stiva si controlul este trecut programului apelant. Acesta elimina parmetriii din stiva readucand stiva in starea din figura (a).
In unele limbaje de programare, exista si varianta apelului prin "copy / restore", caz in care variabila are dublul rol de a transmite o valoare de la programul principal la subrutina si, la terminarea executie procedurii, de a transmite o valoare eventual modificata de la procedura la programul principal.

Principiul RPC (apel sincron)

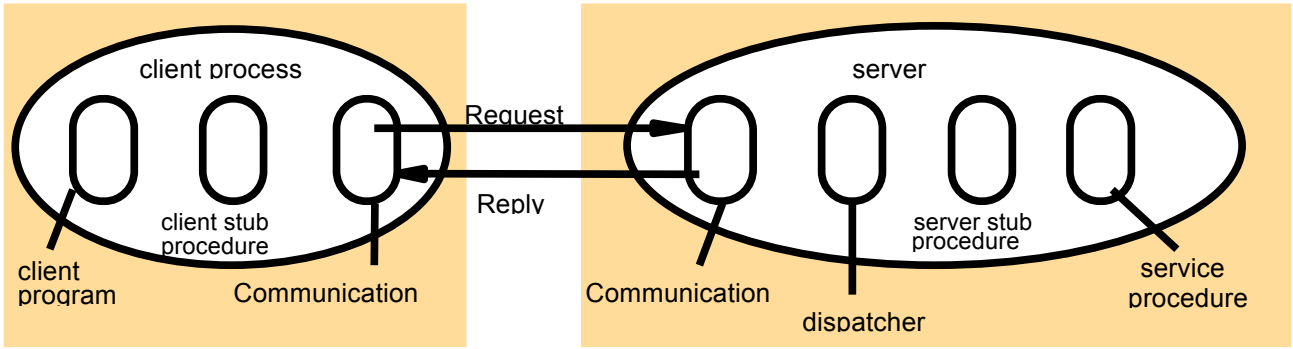
Conceptul a fost introdus de Birrell si Nelson in 1984:



In principiu, cand un proces pe o masina A (client) apeleaza o procedura aflata pe masina B (server), procesul este suspendat si executia continua pe B. La terminarea executiei procedurii, rezultatul este intors procesului. La primirea lui, procesul isi reia executia.

Apel RPC - client

In realitate, la realizarea apelului contribuie mai multe componente ale sistemului, care sunt detaliate in figura urmatoare. Pasii sunt urmatorii:
- programul client apeleaza procedura stub a clientului (care este un proxy al serverului pe masina clientului)
- stub-ul impacheteaza parametrii pentru procedura la distanta, in format standard
- stub-ul apeleaza OS (modulul de comunicare) ca sa transmita mesajul
- asteapta raspunsul



Stub-uri client si server

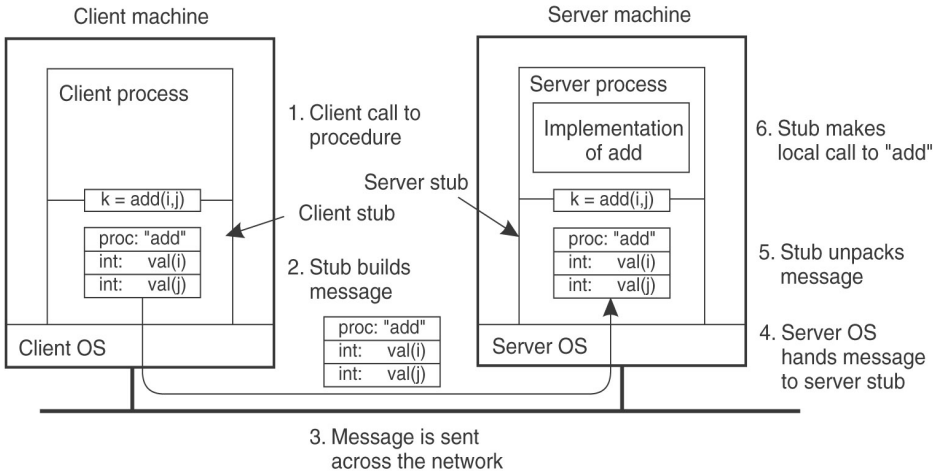
Pe masina server:
- OS apeleaza un stub server care despacheteaza parametrii de la formatul standard la cel al masinii server
- stub-ul server apeleaza procedura server
- serverul umple buf (zona buf este interna stub-ului)
- stub-ul server impacheteaza rezultatul si transmite mesajul stub-ului client.

La client:
- OS da mesajul stub-ului clientului

- stub-ul client despacheteaza, copiaza datele in buf(-ferul) client si intoarce rezultatul.

Executia unui calcul la distanta cu RPC

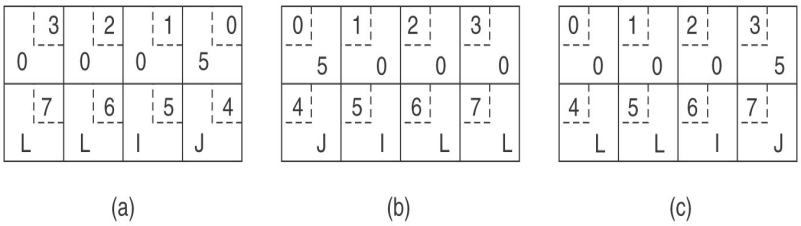
Ce presupune apelul la distanta d.p.d.v. al formatului datelor? Figura urmatoare ilustreaza o situatie simpla in care apelul implica doua masini cu caracteristici hardware si software identice.



Procedura apelata face adunarea a doua numere intregi transmise ca paramatri. Pasi sunt similari descrierii anterioare, figura evidentiind in plus campurile mesajuui transmis intre masina client si masina server, si anume: numele procedurii apelate si valorile celor doi parametri.

Probleme: Reprezentarea datelor

In cazul general, lucrurile sunt mai complicate, deoarece reprezentarile valorilor pe cele doua masini pot fi diferite, iar valorile pot sa nu fie scalare. Figura urmatoare arata cazul in care o valoare se transmite de la o masina Pentium, in care octetii sunt numarati de la dreapta la stanga (reprezentarea little endian), la o masina SPARC, in care octetii sunt numarati de la dreapta la stanga. Deoarece octetii sunt transmisi serial in ordinea adreselor lor, mesajul original de pe Pentium ajunge deformat pw SPARC. Simpla inversare automata a ordinii octetilor fiecarui cuvant nu poate repara eroarea (figura c) deoarece sirurile de caractere nu sunt afectate de ordinea diferita de numarare a octetilor in cele doua reprezentari. Este deci nevoie de o metoda de conversie mai complicata, care sa tina cont de tipurile valorilor transmise.



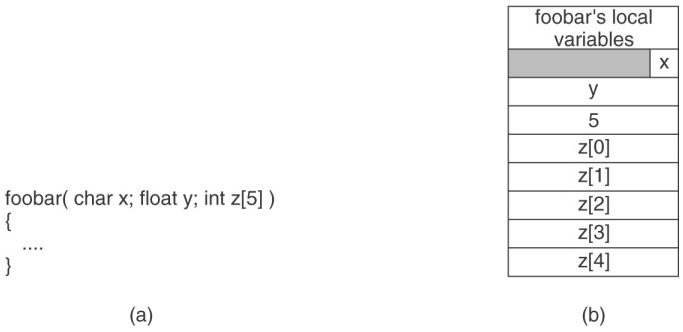
(a) Mesajul original pe Pentium little endian (b) Mesajul receptionat pe SPARC big endian (c) Mesajul dupa inversare

Transferul parametrilor

Solutia de transmitere a parametrilor la distanta se bazeaza pe conversia valorilor de la formatul intern al clientului la un **format de transfer** (operatia se numeste marshalling) si pe conversia de la formatul de transfer la formatul intern al serverului (care poate fi diferit de acela al clientului; operatia se numeste de-marshalling).
Conversia parametrilor valoare este mai simpla si tine seama de reprezentare cum ar fi coduri folosite (de exemplu, Unicode 16 biti pentru caractere sau standardul IEEE #754 pentru numere in virgula mobila) si numararea octetilor (little endian, big endian).
Conversia parametrilor referinta este mai complicata. Regulile pot fi mai usoare pentru structuri simple (de exemplu tablouri, care pot fi transmise ca parametri copy / restore asa cum s-a vazut in exemplul procedurii read dat la inceput). Pentru structuri mai complexe (grafuri), poate fi folosita linearizarea structurii, transmiterea si refacerea ei la celalalt capat. In plus, pentru eficienta comunicatiei, se poate face calificarea parametrilor in, out, in / out care ar scuti conversia si transmiterea lor la apel (parametrii out) sau la returnarea rezultatului (parametrii in).

O procedura si mesajul corespunzator

In figura urmtoare se exemplifica transmiterea parametrilor unei proceduri intr-un format in care: un caracter ocupa octetul din dreapta al unui cuvant, un numar in virgula mobila coupa un cuvant, iar un tablou este un grup de cuvinte precedate de un cuvant care arata



lungimea tabloului.

Specificarea parametrilor

Protocol RPC se refera, in primul rand, la formatul datelor. El defineste modul in care sunt reprezentate diferite tipuri si structuri de date folosind tipuri specifice (cum este cazul XDR -eXternal Data Representation) sau generale (ASN.1—Abstract Syntax Notation). De asemenea, este descrisa semnatura procedurilor apelate intr-un limbaj neutru, de exemplu Interface Definition Language. Aceasta specifica complet procedura si este compilata in stub-urile client si server.
Al doilea element al protocolului este dat de regulile schimbului de mesaje (de ex. TCP).

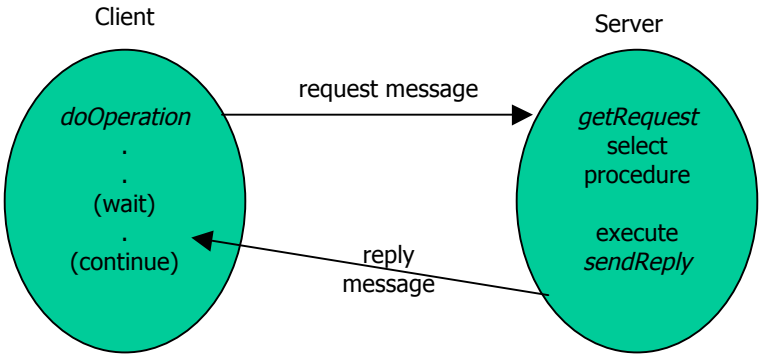
Protocoale RPC – schimbul de mesaje

Modelele folosite in prezenta erorilor de comunicare sunt urmatoarele:

R – request; se foloseste daca procedura nu intoarce o valoare. Clientul nu cere confirmarea executiei procedurii si nu asteapta dupa executia apelului. Este folosit in aplicatii in care erorile sunt rare si nu conteaza in contextul dat.
RR – request/reply este varianta uzuala. In acest caz, raspunsul serveruui este privit ca acknowledge. Un apel ulterior de la client este privit ca o confirmare a primirii raspunsului serverului de catre client.
RRA – request/reply/acknowledge este folosit de server pentru a elimina intrari din istoria pastrata (pentru eventuala re-transmitere a unor raspunsuri pierdute). Un ack poarta un identificator al cererii (requestId). In plus, ack nu blocheaza clientul (acesta nu mai asteapta primirea vreunei confirmari de la server). In fine, ack compenseaza eventuale ack-uri pierdute anterior de server.

Protocol	Client	Server	Client
R	request		
RR	request	reply	
RRA	request	reply	ack

Figura urmatoare arata o implementare posibila a protocolului RR. Clientul lanseaza un apel prin doOperation, care trimite mesajul de cerere la o anumita adresa de socket si se blocheaza in asteaptarea raspunsului, la primirea caruia continua executia. Serverul executa getRequest pentru a prelua cererea impreuna cu adresa socket a clientului, selecteaza procedura, o executa si transmite raspunsul prin sendReply la adresa de socket



a clientului.

Semantica invocarilor RPC

Semantica invocarilor la distanta in cazul defectelor, asa cum este vazuta din partea clientului, include urmatoarele categorii:
Maybe – clientul nu poate spune daca executia procedurii s-a facut sau nu.
Nu se retransmite cererea. Sufera de mai multe tipuri de defecte (omisiune, crash). La timeout, nu se poate identifica defectul (pierdere cerere, raspuns, cadere server).In plus, raspunsul poate veni dupa time-out.
At least once – procedura se executa cel putin o data.
La time-out, se retransmite cererea. Cererea se poate transmite repetat pana cand clientul primeste un rezultat sau un mesaj de eroare. Problema este ca repetarea

executiei procedurii poate conduce la rezultate eronate, cu exceptia cazului in care procedura implementeaza operatii idempotente (care nu schimba starea ca efect al executiei procedurii; de ex. o procedura de citire a timpului curent este idempotenta).

At most once – procedura se executa cel mult o data.

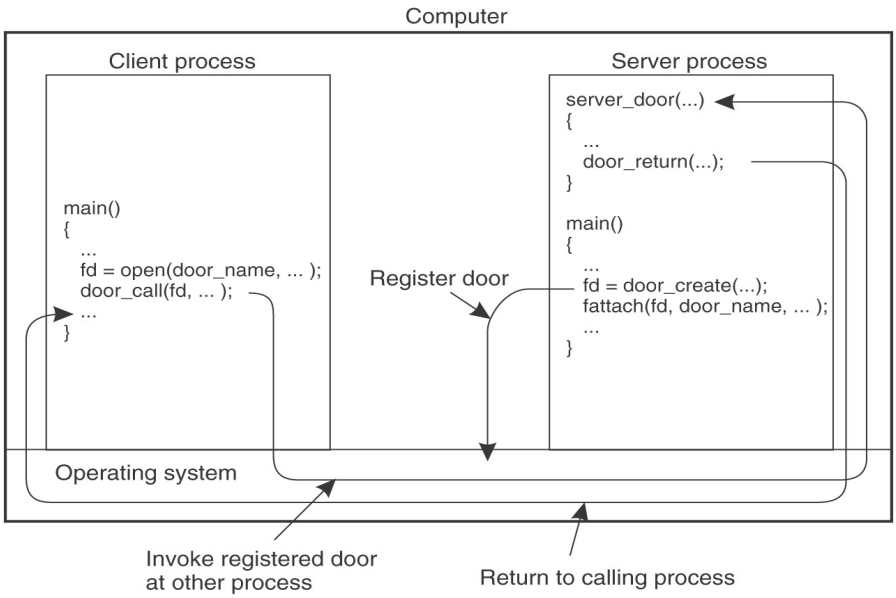
La timeout, se retransmite cererea. Cererea se poate transmite repetat pana cand clientul primeste un rezultat sau un mesaj de eroare. Serverul pastreaza in cache rezultatul executiei procedurii si retransmite rezultatul daca se cere de catre client. Serverul face filtrarea cererilor, adica recunoasterea cererilor noi si a duplicatelor.

Semantica invocarii	retransmite *request*	Filtrare duplicate	Re-executie procedura sau retransmite *reply*
Maybe	Nu	Ne-aplicabil	Ne-aplicabil
At-least-once	Da	Nu	Re-executa procedura
At-most-once	Da	Da	Retransmite *reply*

Porti (doors) ca mecanisme IPC

Mecanismul de doors este folosit cand clientul si serverul stau pe aceeasi masina. Rostul sau este de a face cat mai uniforma executarea procedurii, indiferent daca este un apel local sau la distanta. Portile folosesc facilitati locale de comunicare intre procese, care sunt mai eficiente.

door este o procedura in spatiul serverului care co-locateaza in aceeasi masina cu clientul.



Etapele apelului sunt urmatoarele.

La server:

inregistraeaza door (fd)

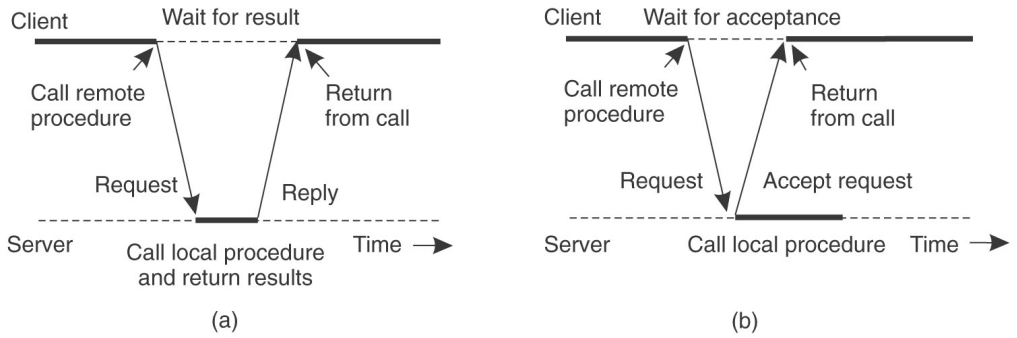
asociaza nume (fattach)

La client:

deschide door (fd)
apeleaza door.

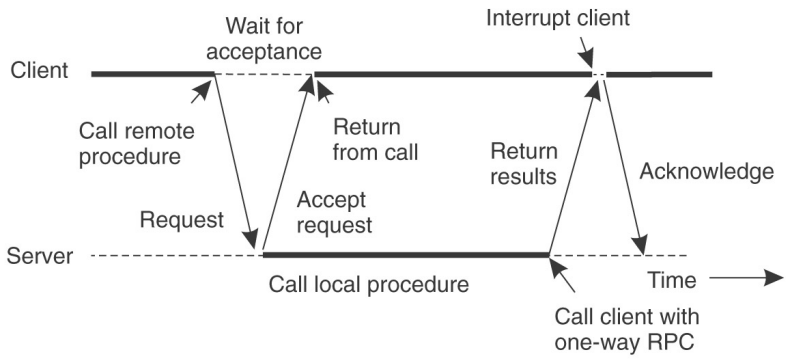
RPC sincron si asincron

Traditional, RPC este sincron (figura a mai jos). In acest caz, clientul isi suspenda executia pana la primirea raspunsului de la server. Exista si alte variante de RPC. In RPC asincron (figura b mai jos) clientul asteapta o confirmare ca serverul a primit cererea si a acceptat executia ei, dar nu mai asteapta raspunsul. Este folosita atunci cand un raspuns nu este necesar. Varianta are o laternativa nummita one-way RPC, in care clientul trimite cererea si continua imediat executia fara a astepta vreo confirmare de la server.



RPC dublu asincron

O alta varianta este RPC dublu asincron. Clientul trimite cererea si obtine confirmarea receptiei de la server, dupa care continua executia. Serverul apeleaza procedura. La terminarea ei, serverul face un apel RPC catre client si transmite rezultatul.



SUN RPC

Numit si **ONC RPC (ONC – Open Network Computing)** a fost printre primele oferte de RPC, la inceputul anilor '80.
Utilizat in scrierea sistemului de fisiere **NFS**.

Poate fi folosit peste **TCP** sau **UDP** la alegere datorita unei biblioteci care implementeaza rutinele de protocoale si sockets pentru suportul la executie a RPC.
Are un **format de codificare a datelor, XDR** (eXternal Data Representation) pentru codificarea datelor transmise intre masini eterogene.
Ofera un **compilator** asociat, numit **rpcgen** care primeste definitia interfetei elaborata de programator si genereaza stub-urile client si server.

XDR – eXternal Data Representation standard

Specificat in **RFC 1014**
Permite definirea:
Procedurilor utilizabile de la distanta
Tipurilor asociate: Constante, typedefs, structuri, uniuni, enumerari.

Caracteristici

- Interfata este identificata prin **nr program** si **nr versiune**
- O definitie de procedura include **semnatura** si **nr procedura**
- Semnatura procedurii include
  tip_rezultat nume_procedura tip_parametru_intrare
- Exista un singur parametru de intrare si un singur rezultat (restrictie eliminata in versiunile urmatoare de RPC)

Exemplu - Airline reservation RPC interface definition

Programatorul include in definitia de interfata: definitii de tipuri de date si declaratiile de proceduri, grupate dupa numere de versiune (pentru a suporta clienti mai vechi care se conecteaza la un server nou), precum si un numar unic de program care permite identificarea interfetei necesare.

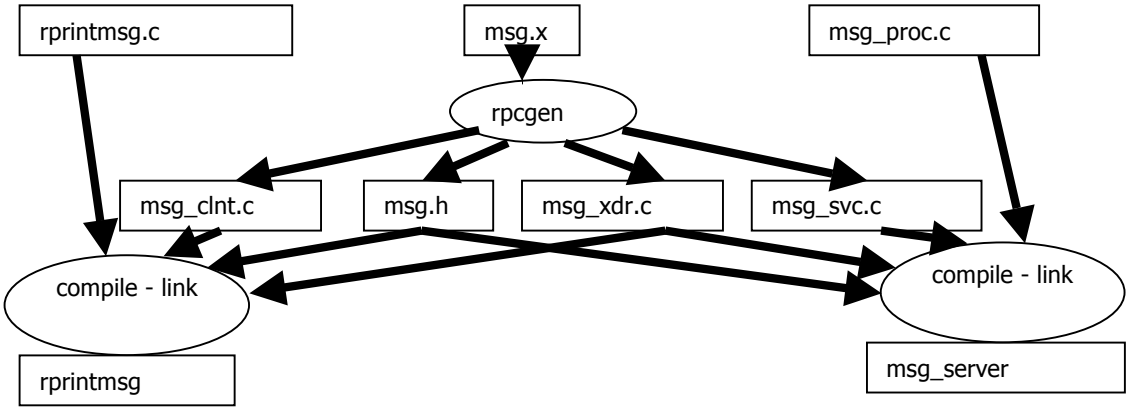
```
struct reserve_args {
  int flight_number;
  string passenger_name<>;
};

struct reserve_result {
  int ok;
  string seat<>;
};

program RESERVATION_PROG {
  version RESERVATION_VERS {
    reserve_result RESERVATION_RESERVE        (reserve_args) = 1;
  } = 1;
} = 400001;
```

rpcgen

Programatorul scrie procedura client (de ex. rprintmsg.c), functia server (msg_proc.c) si specificatia RPC (msg.x).



rpcgen genereaza urmatoarele fisiere (pe baza definitiei din fisierul msg.x):

- procedurile din **stub client** (**msg_clnt.c**)
- procedurile din server: **main, dispecer, stub** (**msg_svc.c**)
- procedurile de **marshalling** si **unmarshalling** XDR (**msg_xdr.c**)

Modificari fata de apel local

Clientul si serverul nu trebuie modificate prea mult fata de varianta unui apel local. Oricum, se adauga actiunile, legate de apelul la distanta, specificate incontinuare.

Clientul

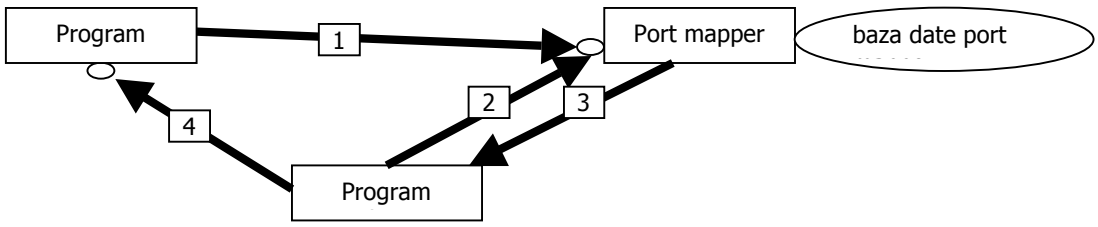
initializeaza interfata RPC cu *clnt_create*
aceasta creaza un RPC handle la programul si versiunea specificate, pe o gazda data; parametri:
nume server
nume si versiune program (generate de rpcgen in msg.h)
protocolul de transport utilizat
apelata inaintea oricarei proceduri la distanta

Server

executa stub si pune procesul in background
creaza un socket si leaga un port local la socket
apeleaza *svc_register* pentru a inregistra portul, numarul si versiunea programului (se contacteaza *port mapper*)
executa *listen* (asteapta cererile clientilor)

Binding

Legarea clientului la server foloseste **port mapper** aflat la un port cunoscut pe fiecare masina care ruleaza RPC. La el se inregistreaza toate **serverele** de pe masina locala prin: **nr program, nr versiune, nr port**



Clientul care cunoaste adresa IP a gazdei serverului apeleaza port mapper pentru afla numarul de port. Secventa este urmatoarea:
1 – serverul creaza port si se inregistreaza la port mapper
2 – clientul cauta serverul dupa numar program si versiune
3 – port mapper transmite numar port
4 – clientul apeleaza serverul
Solutia permite ca serverul sa-si aleaga dinamic numarul de port.

Exemplu

Program de tiparire a datei curente.

date.x

```
/* date.x - description of remote date service */
/* we define two procedures: */
/* bin_date_1 returns the time in binary format */
/* (seconds since Jan 1, 1970 00:00:00 GMT) */
/* str_date_1 converts a binary time to a readable date string */

program DATE_PROG {
version DATE_VERS {
long BIN_DATE(void) = 1; /* procedure number = 1 */
string STR_DATE(long) = 2; /* procedure number = 2 */
} = 1;
} = 0x31415926;
```

server.c

```
#include <rpc/rpc.h>
#include "date.h"
/* bin_date_1 returns the system time in binary format */
/* name obtained from BIN_DATE with lower case and version number 1*/
/* result is a pointer to the value declared in the RPC definition */
long * bin_date_1()
{
static long timeval;     /* must be static!! This value is */
                                 /* used by rpc after bin_date_1 returns */
long time();             /* Unix time function; returns time */
timeval = time((long *)0);
return &timeval;
}
/* str_date_1 converts a binary time into a date string */
char ** str_date_1(bintime)
long *bintime;
{
static char *ptr;        /* return value... MUST be static! */
char *ctime();           /* Unix library function that does the work */
ptr = ctime(bintime);
```

```
return &ptr;
}
```

client.c

```
/* client code */
#include <stdio.h>
#include <rpc/rpc.h>
#include <netconfig.h>
#include "date.h"
main(argc, argv)
int argc;
char **argv;
{
CLIENT *cl;             /* rpc handle */
char *server;
long *lresult;          /* return from bin_date_1 */
char **sresult;                 /* return from str_date_1 */
if (argc != 2) {
fprintf(stderr, "usage: %s hostname\n", argv[0]);
exit(1);
}
server = argv[1];       /* get the name of the server */


/* create the client handle */
if ((cl=clnt_create(server, DATE_PROG, DATE_VERS, "netpath")) ==NULL)
/* netpath - search a NETPATH environment var for a sequence of preferred transport
providers */
{       /* failed! */
        clnt_pcreateerror(server);
        exit(1);
}
/* call the procedure bin_date */
if ((lresult=bin_date_1(NULL, cl))==NULL) {
        /* failed ! */
        clnt_perror(cl, server);
        exit(1);
}
printf("time on %s is %ld\n", server, *lresult);
/* have the server convert the result to a date string */
if ((sresult=str_date_1(lresult, cl)) == NULL) {
        /* failed ! */
        clnt_perror(cl, server);
        exit(1);
}
printf("date is %s\n", *sresult);
```

```
clnt_destroy(cl);       /* get rid of the handle */
exit(0);
```

Suport de securitate

SUN RPC foloseste trei forme de autentificare:

AUTH_NONE fara autentificare
AUTH_UNIX autentificare prin uid si gid
AUTH_DES Secure RPC (descris in detaliu in capitolul despre securitate).

- combinatie chei publice si private (Diffie Hellman + DES)
- presupune autentificarea clientului si serverului

Forma mesajelor SUN RPC

Call	**Reply**
Xid Call / reply Rpc version Program # Program version Procedure # Auth stuff Arguments	Xid Call / reply Accepted? (vs bad rpc version or auth failure) Auth stuff Success? (vs bad prog / proc #) results

Reprezentarea datelor pentru XDR

int	complement fata de 2, 32 biti, little endian
unsigned int	32 biti little endian
float	32 biti: semn, exponent pe 8 biti, mantisa pe 23 biti
double	64 biti: semn, exponent pe 11 biti, mantisa pe 52 biti
string	lungime sir pe 4 octeti plus (n+r) octeti pentru sir umplut la multiplu de 4 octeti
tablou	lungime fixa: n elemente de la 0 la n-1
	lungime variabila: lungime pe 4 octeti urmata de n elemente de la 0 la n-1
struct	componentele in ordinea declararii
union	discriminant pe 4 octeti urmat de valoarea de pe varianta selectata
void	0 octeti

DCE RPC

Dezvoltat de Open Software Foundation (acum The Open Group). A fost proiectat pentru Unix si extins la alte OS (VMS, Windows NT). este bazat pe modelul client-server.
DCE RPC defineste si servicii client-server (incluse in DCE RPC):

Distributed file service
Directory service
Security service
Distributed time service

Obiective DCE RPC

Obiectivele DCE RPC sunt :clasice":

- Transparenta locatiei
- Transport mesaje intre client si server
- Conversie tip date
- Transparenta limbajului (ex. client Java, server C)
- Transparenta SO
- Transparenta hardware
- Transparenta protocoalelor de retea.

Caracteristici

Este similar Sun RPC prin urmatoarele elemente:

- Interfete scrise in *Interface Definition Notation (IDN)*
- Un compilator IDN genereaza header-ul + stub-urile client si server

Diferentele se manifesta la identificarea serverului:

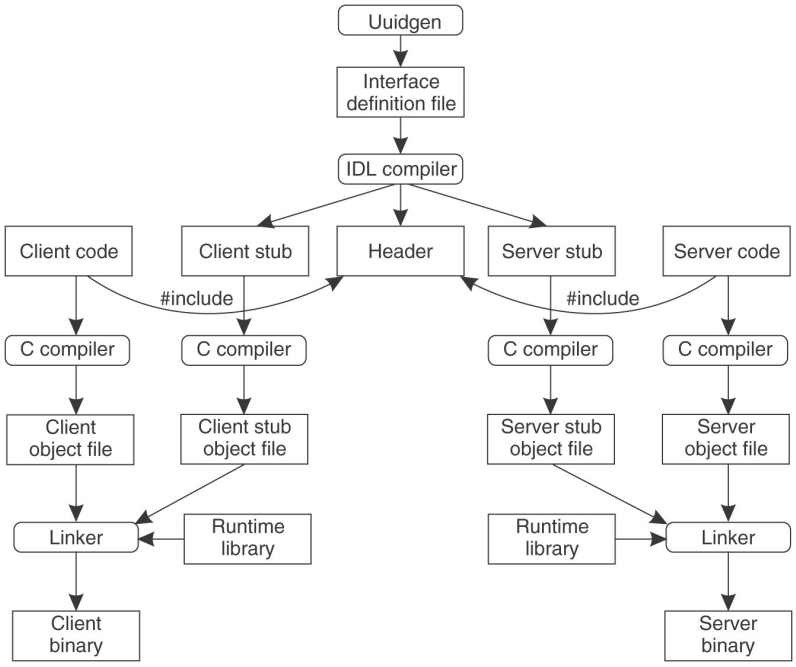
- daca la Sun identificarea se face printr-un numar unic de 32 de biti ales de programator,
- la DCE RPC ID unic are 128 biti si este generat prin programul *uuidgen* care genereaza si fisierul prototip IDN.

Scriere client si server in DCE RPC

Pasii:

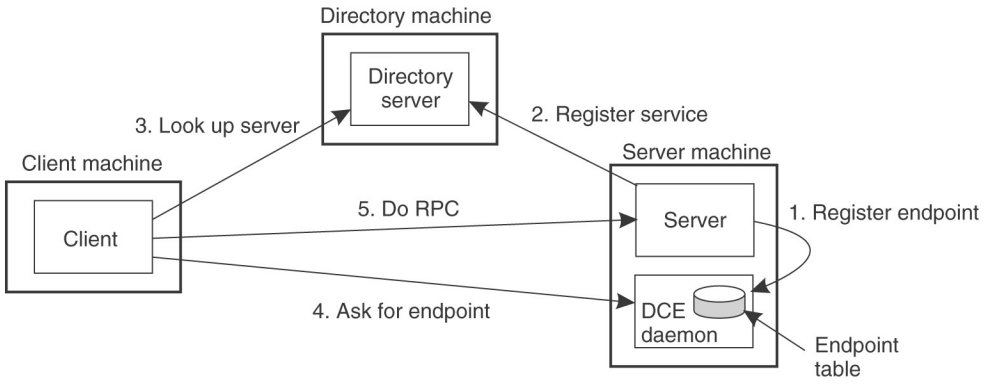
1 – un appel **uuidgen** creaza un fisier prototip IDN in care programatorul adauga specificatia de interfata

2 – apelul compilatorului IDL produce codul pentru stub-urile clienta si server, precum si un fisier antet care este apoi inclus in codul client si in codul server pentru compilare.



Legare client - server in DCE

La Sun – programatorul trebuie sa cunoasca masina serverului(adresa IP). In DCE, serverul inregistreaza endpoint cu un DCE RPC daemon local (pasul 1 in figura urmatoare) si corespondenta {nume_server, masina} cu un server de directoare (pasul 2).



Ca urmare, descoperirea serverului se face in urmatoarele etape: clientul cauta adresa serverului cu un serviciu de directoare (pasul 3), regaseste endpoint cu DCE daemon (pasul 4) si apoi face apelul RPC (pasul 5).

Semantici RPC in prezenta defectelor

Scopul RPC este ascunderea localizarii procedurii, astfel incat apelul la distanta sa arate ca unul local. Transparenta este amenintata de posibila aparitie a defectelor. Defectele de comunicare – pierderea mesajelor (defecte prin omisiune) – sunt ascunse clientului prin utilizarea unui transport sigur TCP. Defectele de crash – canalul, clientul, serverul - nu sunt mascate. Analizam mai multe clase de defecte.

Clientul nu poate localiza serverul

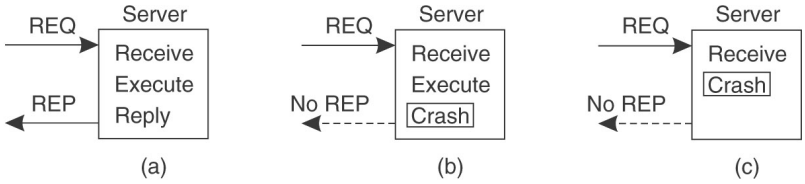
Aces lucru se intampla daca server oprit sau modificat, de exemplu are o alta interfata. Solutii le posibile sunt provoaca exceptie (Java) sau folosirea unui handler de semnalizare (in C). Neajunsul este ca nu orice limbaj are exceptii sau handler de semnalizare. In plus, este distrusa transparenta, deoarece aceste mecanisme nu se regasesc in apelurile locale.

Mesaj de cerere pierdut

Solutia este folosirea unui timer in OS client, care sa anunte clientul cand sa re-transmita cererea. Solutia "merge" daca mesajul de cerere a fost cu adevarat pierdut. Daca cererea nu este pierduta, serverul ar trebui sa faca diferenta intre cererea originala si copie. Ca alternativa, serverul ar putea re-executa invocarea daca operatia invocata este idempotenta.

Caderea Serverului

Secventa normala de operare a serverului este prezentata in figura urmatoare (cazul a): cererea este primita, executata si raspunsul este trimis. Este posibil (cazul b) ca serverul sa cada dupa executia operatiei daqr inainte de transmiterea raspunsului. In fine (cazul c), serverul poate pica dupa primirea cererii, dar inainte de executia procedurii.



Problema este ca, atunci cand apare un timeout, clientul nu poate diferentia intre b si c.
Solutiile pentru client sunt urmatoarele:
- repeta cererea pana reuseste (at least once)
- semnaleaza eroarea (at most once)
- nimic (maybe)
- solutie teoretica? (exactly once); din pacate, se arata (in continuare) ca solutia nu este posibila.

Exactly once - Strategii client si server

Consideram un sistem client-server pentru tiparire text.
Strategiile serverului sunt:

sa trimita ACK inainte de a spune printerului sa tipareasca
sa trimita ACK dupa tiparirea textului.

Presupunem ca serverul anunta ca a cazut si acum este iar operational.
Strategia de retransmitere a cererii de catre client poate fi una din:
mereu
niciodata
doar cand nu s-a confirmat trimiterea cererii la server
doar cand s-a confirmat cererea de tiparire.

Combinatii

In analiza combinatiilor posibile, consideram urmatoarele evenimente:

transmite mesaj terminare (M)
tipareste text (P)
cadere server (C)

Combinatiile posibile sunt urmatoarele:

M P C - caderea are loc dupa transmiterea mesajului de terminare si tiparirea textului
M C (P) - caderea are loc dupa transmiterea mesajului de terminare si inainte de tiparirea textului
P M C - caderea are loc dupa tiparirea textului si transmiterea mesajului de terminare
P C (M) - caderea are loc dupa tiparirea textului si inainte de transmiterea mesajului de terminare
C (M P) - caderea are loc inainte ca serverul sa faca ceva
C (P M) - caderea are loc inainte ca serverul sa faca ceva.

unde intre paranteze au fost mentionate evenimentele care nu mai pot avea loc din cauza caderii serverului.

Client	Server					
	Strategy M → P			Strategy P → M		
Reissue strategy	MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
Always	DUP	OK	OK	DUP	DUP	OK
Never	OK	ZERO	ZERO	OK	OK	ZERO
Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
Only when not ACKed	OK	ZERO	OK	OK	DUP	OK

OK = Text is printed once
DUP = Text is printed twice
ZERO = Text is not printed at all

In figura anteriaora, au fost incluse toate combinatiile posibile. Dupa cum se vede nici o combinatie de strategii client si server nu lucreaza corect in toate combinatiile posibile de evenimente.

Pierdere mesaje reply
Solutia este folosirea unui timer. Ea merge pentru operatii idempotente. Pentru celelalte, clientii asociaza cererilor numere de secventa cererilor si serverul tine evidenta raspunsurilor asociate. Ca masura suplimentara, un bit in mesajele de cerere poate face distinctia intre original si copii.

Cadere Client
Cand un client cade, calculele in curs devin orfane: nici un parinte (client) nu asteapta rezultatele lor. Calculele orfane pun probleme de: consum inutil de resurse si de fisiere incuiate (lock). In plus, apar probleme de sincronizare cu un client cazut si reactivat (re-boot-at).
Solutiile posibile sunt urmatoarele:
exterminare (log entry)
Presupune inregistrarea fiecarui apel RPC intr-o memorie care rezista caderilor clientului. Cand clientul cade si apoi se recupereaza, el inspecteaza log-ul si trimite mesaje de exterminare a orfanilor.
Probleme: inregistrarea fiecarui apel este costisitoare; descendentii orfanilor (grandorfans) sunt greu, daca nu imposibil, de localizat; in cazul unei retele partitionate nu pot fi identificati toti orfanii.
reincarnare (epoci)
Timpul este divizat in epoci numerotate secvential. Un client rebootat declara inceputul unei noi epoci si anunta aceasta printr-un broadcast. Toate calculele din epoci anterioare sunt distruse. Raspunsurile din epoci anterioare care totusi sosesc la client sunt ignorate de client.
reincarnare lina (localizare proprietar)
Similara cu reincarnarea, doar ca la primirea anuntului de epoca noua, inainte de distrugere se incearca identificarea proprietarului. Calculul este distrus doar daca proprietarul nu poate fi gasit.
expirare (bucata de timp T standard)
Fiecarui apel RPC i se acorda un interval de timp T standard pentru a fi efectuat. Daca executia nu poate fi terminata, serverul cere explicit un alt interval T. Daca

nu primeste confirmarea executia este terminata. Pentru a folosi acest mecanism, re-boot-area unui client dupa cadere se face dupa trecerea unui interval de timp T, in care toate calculele initiate anterior sunt terminate. Problema este alegerea corespunzatoare a lui T.

Concluzii

Conceptul RPC a fost extins la obiecte (DCOM, CORBA, Java RMI).
Aspectele de semantica in caz de defectare raman valide in aceste noi modele.

Bibliografie

A.S. Tanenbaum, M. van Steen. Distributed Systems. Principles and paradigms, Prentice Hall 2007

George Coulouris, Jean Dollimore,Tim Kindberg. Distributed Systems Concepts and Design (Third Edition), Addison-Wesley, 2001